INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

GRUPO 3CV4

GESTOR DE CITAS MÉDICAS

Entrega Parcial 1 · Etapa Funcional

Materia: Paradigmas de Programación
Profesora: Martínez Hernández Guadalupe Ana Gabriela

Integrantes

Contreras Cruz Joseph
Mendez Gamboa Santiago
Rosas Rios Luis Gabriel

NOTA DE ACTUALIZACIÓN · SEGUNDO PARCIAL

Alcance del MVP evaluable

Esta nota acompaña al reporte original del primer parcial, que se conserva íntegro. Sus requerimientos R01–R14 y casos de uso describen la visión general del sistema; no todos son funciones terminadas. Para el segundo parcial solo se presentan como implementadas y demostrables las tres operaciones siguientes:

01 CONSULTAR DISPONIBILIDAD

Seleccionar médico, fecha y duración; mostrar horarios libres y ocupados.

02 AGENDAR UNA CITA

Registrar una reserva para un paciente ficticio y rechazar empalmes.

03 CANCELAR UNA CITA

Marcar una cita como cancelada y liberar el horario que ocupaba.

Fuera del alcance de esta demostración

Reprogramar, crear o modificar médicos, cuentas y roles, notificaciones, historias clínicas, edición independiente de pacientes y otras funciones proyectadas en la primera etapa. No se presentan como terminadas.

La continuación del segundo parcial, al final de este PDF, documenta el diseño del MVP, el diagrama de clases, los archivos ejecutables y las pruebas. Las tres pruebas del laboratorio serán elegidas por la profesora.

1. Recuperación del trabajo previo

Para esta primera entrega retomamos la idea que traíamos desde el avance anterior: un gestor de citas médicas. La intención es que el sistema pueda guardar y organizar la información de pacientes, doctores, horarios y las citas como tal. Con eso cubrimos las operaciones que normalmente se esperarían de una herramienta así: registrar, consultar, agendar, cancelar y reprogramar, además de mandar confirmaciones y cuidar los datos personales.

En cuanto a las tecnologías, no nos movimos mucho respecto a lo que ya habíamos planteado. Python se queda como lenguaje principal porque es multiparadigma y eso nos deja usar el enfoque que más nos convenga en cada parte. JavaScript lo usamos del lado del navegador, SQL queda para la persistencia y HTML/CSS para la estructura y presentación de la página.

1.1 Requerimientos

ID	Requerimiento	Actor
R01	Registrar pacientes en el sistema	Recepción
R02	Registrar doctores y sus especialidades	Administrador
R03	Permitir iniciar sesión a los usuarios	Usuarios autorizados
R04	Mostrar los horarios disponibles de los doctores	Recepción / Paciente
R05	Permitir agendar una cita médica	Recepción
R06	Impedir agendar una cita cuando el doctor ya tenga otra cita en ese horario	Recepción
R07	Permitir cancelar una cita	Recepción / Administrador
R08	Permitir reprogramar una cita	Recepción / Administrador
R09	Mostrar al paciente sus citas próximas	Paciente
R10	Permitir al doctor consultar su agenda	Doctor
R11	Permitir al administrador agregar, modificar o eliminar usuarios	Administrador
R12	Enviar una confirmación cuando se registre una cita	Automático
R13	Mantener protegidos los datos personales de pacientes y doctores	Sistema
R14	Permitir al doctor notificar que no asistirá para cancelar su lista	Doctor

1.2 Roles y actores

Actor	Responsabilidades principales

Actor	Responsabilidades principales
Administrador	Gestiona usuarios y operaciones administrativas.
Doctor	Consulta agenda y horarios; puede notificar que no asistirá.
Paciente	Consulta sus citas próximas y la información necesaria para su atención.
Recepción	Registra pacientes y realiza el agendado y gestión de citas.
Personal autorizado	Accede a funciones asignadas por el sistema.

1.3 Casos de uso

ID	Caso de uso	Actor principal
CU01	Registrarse como paciente	Recepción / Paciente
CU02	Registrarse como doctor	Administrador
CU03	Consultar horarios disponibles	Recepción / Paciente
CU04	Agendar cita	Recepción
CU05	Cancelar cita	Recepción / Administrador
CU06	Reprogramar cita	Recepción / Administrador
CU07	Consultar calendario de citas	Doctor / Recepción
CU08	Consultar doctores disponibles	Paciente / Recepción
CU09	Administrar usuarios	Administrador
CU10	Modificar datos del paciente	Recepción / Administrador
CU11	Envío de notificación al paciente y al doctor	Sistema

1.4 Elección y evaluación de lenguajes

Elegimos Python como lenguaje principal por varias razones. La primera es que permite mezclar sin problema programación imperativa, orientada a objetos y funcional dentro del mismo proyecto, que es justo lo que necesitábamos para esta etapa. A eso se suma que la sintaxis es bastante legible, el tipado es dinámico pero fuerte, no tenemos que preocuparnos por administrar memoria y podemos tratar las funciones como cualquier otro valor. Tampoco hay que olvidar que tiene un ecosistema muy amplio para aplicaciones web, bases de datos y pruebas, lo cual nos ahorra bastante trabajo.

JavaScript lo vamos a usar del lado del frontend: eventos, validaciones del formulario y actualizaciones dinámicas de la interfaz. SQL queda reservado para la persistencia relacional y HTML/CSS para la estructura y el diseño visual.

1.5 Distribución propuesta del sistema

Tecnología	Responsabilidad

Tecnología	Responsabilidad
Python	Lógica del servidor, reglas de negocio, autenticación y validación funcional.
JavaScript	Interacción del frontend, formularios, selección de horarios y consumo de la API.
SQL	Persistencia futura de usuarios, médicos, pacientes, horarios y citas.
HTML/CSS	Estructura y diseño visual de la aplicación web.

2. Selección de una funcionalidad

2.1 Funcionalidad seleccionada: búsqueda de horarios disponibles de un médico

De todo el sistema, la funcionalidad que decidimos desarrollar a fondo fue la búsqueda de horarios disponibles de un médico. La función recibe la agenda del médico, su horario laboral y la duración que se quiere para la cita, y con eso calcula los espacios libres donde esa cita podría colocarse sin empalmarse con otra. La elegimos porque era justo la que ya habíamos empezado a trabajar en el pseudocódigo del avance previo y porque se presta muy bien para mostrar el paradigma funcional.

El problema que resuelve es bastante concreto: evita que recepción o el paciente tengan que ponerse a revisar cita por cita para encontrar un hueco libre. En lugar de eso, la función toma la lista de citas y devuelve directamente una lista de intervalos disponibles.

Los usuarios que la consultarían son Recepción y Paciente, dentro del caso de uso CU03. Para esta etapa usamos una agenda ficticia como entrada; todavía no hace falta base de datos.

2.2 Relación con requerimientos y casos de uso

Elemento	Relación
R04	El resultado principal de la funcionalidad son los horarios disponibles que el sistema muestra al usuario.
R06	La disponibilidad calculada evita seleccionar un horario que ya esté ocupado por otra cita.
CU03	Es el caso de uso principal: consultar horarios disponibles.
CU04	La información que produce esta funcionalidad se puede usar después para agendar una cita.

2.3 Entradas, transformaciones y salidas

Etapa	Contenido
Entradas	Lista de citas con inicio y duración; horario laboral; duración solicitada para una nueva cita.
Validación	Se comprueba que la duración sea positiva y que las citas existentes permanezcan dentro del horario laboral.

Etapa	Contenido
Transformación 1	Cada cita se convierte en un intervalo [inicio, fin].
Transformación 2	Los intervalos se ordenan por inicio y los que se traslapan se fusionan.
Transformación 3	Se calculan los huecos entre los intervalos ocupados.
Transformación 4	Cada hueco se divide en bloques consecutivos de la duración solicitada.
Salida	Lista de intervalos disponibles o un resultado de error.

2.4 Adecuación al paradigma funcional

Esta funcionalidad se presta bastante al paradigma funcional porque se puede ver como una cadena de transformaciones de datos. Cada función recibe lo que necesita de manera explícita y devuelve estructuras nuevas, sin depender de nada externo ni modificar lo que le llegó. Eso hace más fácil razonar sobre cada paso por separado y reutilizar funciones pequeñas en otros lados.

Comparado con una solución imperativa, nos parece que aquí ganamos en varias cosas: casi no usamos variables mutables, se ve claro dónde termina una transformación y empieza otra, podemos usar composición y funciones de orden superior, y cada función se puede probar sola sin montar todo el flujo. Como desventaja, crear colecciones nuevas en cada paso puede pesar más en memoria si los datos fueran enormes, y el estilo puede resultar raro para quien siempre ha programado con ciclos y variables que se van modificando. Para esta etapa, con datos ficticios y pequeños, eso no representa un problema real.

3. Diseño de la solución

3.1 Estructuras inmutables

Se utilizan tres estructuras principales. Cita representa una cita existente; Intervalo representa un periodo de tiempo; Resultado permite devolver Éxito o Error. En Python se implementan con dataclasses congeladas mediante @dataclass(frozen=True), evitando la modificación posterior de sus atributos.

Tipo	Campos	Propósito
Cita	inicio, duracion	Representar una cita de la agenda.
Intervalo	inicio, fin	Representar tiempo ocupado o disponible.
Resultado	Exito(valor) / Error(mensaje)	Comunicar una salida válida o un error sin excepciones para reglas esperadas.

3.2 Funciones, entradas y salidas

Función	Entrada	Salida
calcular_fin	Cita	Entero
validar_cita	Cita, Intervalo	Resultado<cita>
cita_a_intervalo	Cita	Intervalo

Función	Entrada	Salida
ordenar_por_inicio	Lista de intervalos	Nueva lista ordenada
fusionar	Lista de intervalos	Nueva lista fusionada
calcular_huecos	Lista de intervalos, horario	Lista de huecos
dividir_hueco	Intervalo, duración	Lista de intervalos
formatear_hora	Minutos	Texto HH:MM
buscar_disponibilidad	Citas, horario, duración	Resultado<lista<intervalo>>

3.3 Pseudocódigo

```
TIPO Cita = (inicio: Entero, duracion: Entero)

TIPO Intervalo = (inicio: Entero, fin: Entero)

TIPO Resultado = Exito(valor) | Error(mensaje)

CONSTANTE HORARIO_LABORAL = Intervalo(480, 840)


FUNCION calcularFin(cita) -> Entero

    RETORNAR cita.inicio + cita.duracion

FIN FUNCION


FUNCION validarCita(cita, horario) -> Resultado

    SI cita.duracion <= 0 ENTONCES

        RETORNAR Error("Duracion invalida")

    SI cita.inicio < horario.inicio O calcularFin(cita) > horario.fin ENTONCES

        RETORNAR Error("Cita fuera del horario laboral")

    RETORNAR Exito(cita)

FIN FUNCION


FUNCION citaAIntervalo(cita) -> Intervalo

    RETORNAR Intervalo(cita.inicio, calcularFin(cita))

FIN FUNCION


FUNCION ordenarPorInicio(intervalos) -> Lista

    RETORNAR copiaOrdenada(intervalos, clave = lambda i -> i.inicio)

FIN FUNCION


FUNCION fusionar(intervalos) -> Lista

    FUNCION acumular(acc, actual) -> Lista

        SI acc esta vacia ENTONCES

            RETORNAR [actual]
```

```
        ultimo <- ultimo elemento de acc
        SI actual.inicio <= ultimo.fin ENTONCES
            combinado <- Intervalo(ultimo.inicio, max(ultimo.fin, actual.fin))
            RETORNAR (acc sin ultimo) + [combinado]
        RETORNAR acc + [actual]
    FIN FUNCION
    RETORNAR reducir(intervalos, acumular, [])
FIN FUNCION

FUNCION calcularHuecos(ocupados, horario) -> Lista
    puntos <- [Intervalo(horario.inicio, horario.inicio)] + ocupados
              + [Intervalo(horario.fin, horario.fin)]
    pares <- zip(puntos sin ultimo, puntos sin primero)
    candidatos <- mapear(pares, lambda (a,b) -> Intervalo(a.fin, b.inicio))
    RETORNAR filtrar(candidatos, lambda h -> h.fin > h.inicio)
FIN FUNCION

FUNCION dividirHueco(hueco, duracion) -> Lista
    cantidad <- division_entera(hueco.fin - hueco.inicio, duracion)
    RETORNAR mapear(rango(0, cantidad),
                    lambda k -> Intervalo(hueco.inicio + k*duracion,
                                          hueco.inicio + (k+1)*duracion))
FIN FUNCION

FUNCION buscarDisponibilidad(citas, horario, duracion) -> Resultado
    validar duracion
    validaciones <- mapear(citas, lambda c -> validarCita(c, horario))
    SI existe error EN validaciones -> RETORNAR primer error
    espacios <- componer(
        lambda cs -> mapear(cs, citaIntervalo),
```

```
        ordenarPorInicio,
        fusionar,
        lambda ocs -> calcularHuecos(ocs, horario),
        lambda hs -> aplanar(mapear(hs, lambda h -> dividirHueco(h, duracion)))
    )(citas)
    RETORNAR Exito(espacios)
FIN FUNCION
```

3.4 Conceptos funcionales aplicados

- Funciones puras: el resultado depende solo de las entradas, no de estado externo.
- Inmutabilidad: las citas y los intervalos originales no se modifican; siempre se producen colecciones nuevas.
- Funciones de orden superior: map, filter, reduce, existe y componer reciben funciones como parámetro.
- Lambdas: las usamos como claves, filtros y transformaciones pequeñas.
- Composición: buscar_disponibilidad encadena varias etapas de transformación.
- Separación cálculo / E-S: la lógica de negocio quedó aparte de la impresión y de la interfaz web.

4. Implementación

4.1 Organización del proyecto

```
gestor_citas_project/
├── app.py
├── INICIAR.bat
├── abrir_frontend.bat
├── ejecutar_pruebas.bat
├── EJECUTAR.txt
├── README.md
├── data/
│   └── citas_ficticias.json
├── frontend/
│   ├── index.html
│   ├── styles.css
│   └── app.js
├── src/
│   ├── __init__.py
│   ├── gestor_citas.py
│   └── servidor.py
├── tests/
```

```
│   ├── __init__.py
│   └── test_gestor_citas.py
└── docs/
```

4.2 Ejecución en Python y Visual Studio Code

1. Abrir la carpeta completa del proyecto en Visual Studio Code.
2. Abrir Terminal > Nueva terminal.
3. Ejecutar python app.py. En Windows también se puede ejecutar py app.py.
4. El programa inicia un servidor local y abre el navegador en una dirección 127.0.0.1, buscando automáticamente otro puerto si el 8000 ya está ocupado.
5. En VS Code también existe una configuración de ejecución llamada “Gestor de Citas - Web”.
6. Las pruebas se ejecutan con python -m unittest discover -s tests -v o mediante la configuración “Pruebas - unittest”.

4.3 Frontend desarrollado

Armamos una interfaz web sin dependencias externas. El frontend usa HTML, CSS y JavaScript puro, y presenta un flujo sencillo para consultar disponibilidad: se elige médico, fecha y duración, se ejecuta la búsqueda y se muestran las citas ocupadas junto con los horarios disponibles para seleccionar uno.

La interfaz consume dos rutas del servidor: GET /api/doctores para traer el catálogo ficticio y GET /api/disponibilidad?doctor=...&duracion=... para pedir el cálculo que hace Python.

Como respaldo para pruebas de interfaz, index.html también tiene un modo de demostración local. Si se abre directamente o con Live Server y no hay conexión con Python, se cargan los mismos datos ficticios y el frontend puede mostrar resultados sin depender del servidor.

4.4 Datos ficticios

ID	Médico	Especialidad	Citas iniciales
MED001	Dra. Ana Lopez	Medicina general	09:00-09:30; 10:00-11:00; 10:30-11:00
MED002	Dr. Carlos Martinez	Cardiología	08:00-08:30; 09:30-10:00; 12:00-13:00
MED003	Dra. Sofia Hernandez	Pediatría	Sin citas

5. Pruebas

Este apartado documenta las pruebas realizadas al frontend del Gestor de Citas Médicas, abierto con el archivo abrir_frontend.bat. Ese archivo abre frontend/index.html directamente en el navegador, sin servidor de Python, por lo que la aplicación trabaja en modo demostración con la lógica de app.js (función fallbackAvailability).

Metodología

- **Cantidad:** 20 casos de prueba.

- **Tipos de caso:** Normales (P01–P03, P13), límites (P04, P07, P09), datos vacíos (P03, P15, P16, P19) e inválidos (P05, P06, P08, P18), además de casos de interfaz (P12–P19).
- **Resultado esperado antes de ejecutar:** Los valores esperados se calcularon a mano (horario laboral 08:00–14:00, es decir 480–840 minutos) y se registraron en la Fase 1 antes de correr cualquier prueba (Figura 1).
- **Ejecución:** La Fase 2 ejecuta cada caso contra el código real, registra el resultado obtenido y lo compara con el esperado para marcarlo como Aprobada o Fallida (Figura 2). Los casos de interfaz se verificaron además en un navegador real (Chromium) abriendo index.html como archivo local, con una captura por caso.

Los resultados completos se presentan en la tabla de la siguiente página.

Casos de prueba y resultados

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
P01	Disponibilidad MED001, bloques de 30 min (normal)	citas 540/30, 600/60, 630/30; duración 30	n=9; primero=08:00-08:30; ultimo=13:30-14:00	n=9; primero=08:00-08:30; ultimo=13:30-14:00	**Aprobada**
P02	Disponibilidad MED002, bloques de 30 min (normal)	citas 480/30, 570/30, 720/60; duración 30	n=8; primero=08:30-09:00; ultimo=13:30-14:00	n=8; primero=08:30-09:00; ultimo=13:30-14:00	**Aprobada**
P03	Agenda vacía (dato vacío), bloques de 60 min	citas = []; duración 60	n=6; primero=08:00-09:00; ultimo=13:00-14:00	n=6; primero=08:00-09:00; ultimo=13:00-14:00	**Aprobada**
P04	Duración cero (límite inválido)	citas = []; duración 0	error=Duracion invalida	error=Duracion invalida	**Aprobada**
P05	Duración negativa (inválida)	citas = []; duración -15	error=Duracion invalida	error=Duracion invalida	**Aprobada**
P06	Duración mayor que la jornada (361 min)	citas = []; duración 361	error=La duracion excede el horario laboral	error=La duracion excede el horario laboral	**Aprobada**
P07	Duración igual a la jornada completa (límite válido)	citas = []; duración 360	n=1; primero=08:00-14:00; ultimo=08:00-14:00	n=1; primero=08:00-14:00; ultimo=08:00-14:00	**Aprobada**
P08	Cita que se pasa del cierre (inicio 830, dur 20 → fin 850)	citas 830/20; duración 30	error=Cita fuera del horario laboral	error=Cita fuera del horario laboral	**Aprobada**
P09	Cita que termina justo al cierre (límite válido, fin = 840)	citas 810/30; duración 30	n=11; primero=08:00-08:30; ultimo=13:00-13:30	n=11; primero=08:00-08:30; ultimo=13:00-13:30	**Aprobada**
P10	Citas traslapadas (540-600 y 570-600)	citas 540/60, 570/30; duración 30	n=10; primero=08:00-08:30; ultimo=13:30-14:00	n=10; primero=08:00-08:30; ultimo=13:30-14:00	**Aprobada**
P11	Citas desordenadas: resultado correcto y lista de entrada sin cambios	citas [630/30, 540/30]; duración 30	n=10; primero=08:00-08:30; ultimo=13:30-14:00; entradaIntacta=true	n=10; primero=08:00-08:30; ultimo=13:30-14:00; entradaIntacta=true	**Aprobada**
P12	Abrir el frontend sin servidor: carga de médicos en modo demostración	abrir index.html; fetch("/api/doctores") falla	medicos=3; primero=MED001; conexion=Modo demostración: frontend sin servidor.	medicos=3; primero=MED001; conexion=Modo demostración: frontend sin servidor.	**Aprobada**
P13	UI: buscar MED001, 30 min, fecha 2026-10-05	médico MED001; duración 30; fecha 2026-10-05; clic en "Buscar horarios"	horarios=9; contador=9 disponibles; ocupadas=3; mensaje=Fecha: 2026-10-05 · Duración: 30 minutos · Modo demostración	horarios=9; contador=9 disponibles; ocupadas=3; mensaje=Fecha: 2026-10-05 · Duración: 30 minutos · Modo demostración	**Aprobada**
P14	UI: MED001 con 120 min (solo cabe un hueco)	médico MED001; duración 120	horarios=1; contador=1 disponibles; inicioPrimero=11:00	horarios=1; contador=1 disponibles; inicioPrimero=11:00	**Aprobada**
P15	UI: médico sin citas (MED003, agenda vacía), 30 min	médico MED003; duración 30	horarios=12; ocupadas=0; textoOcupadas=No hay citas ocupadas. Todo el horario laboral está libre.	horarios=12; ocupadas=0; textoOcupadas=No hay citas ocupadas. Todo el horario laboral está libre.	**Aprobada**
P16	UI: fecha vacía (dato vacío)	médico MED001; duración 30; fecha = ""	mensaje=Fecha: sin fecha · Duración: 30 minutos · Modo demostración; horarios=9	mensaje=Fecha: sin fecha · Duración: 30 minutos · Modo demostración; horarios=9	**Aprobada**

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
P17	UI: seleccionar un horario con clic	MED001, 30 min; clic en el primer horario (08:00)	mensaje=Horario seleccionado: 08:00.; marcado=true	mensaje=Horario seleccionado: 08:00.; marcado=true	**Aprobada**
P18	UI: médico inexistente (dato inválido)	médico = "MED999"; duración 30	mensajeDeError=true; horarios=0; botonHabilitado=true	mensajeDeError=true; horarios=0; botonHabilitado=true	**Aprobada**
P19	UI: sin médico seleccionado (valor vacío)	médico = ""; clic en "Buscar horarios"	mensaje=Los horarios disponibles aparecerán aquí.; horarios=0; botonHabilitado=true	mensaje=Los horarios disponibles aparecerán aquí.; horarios=0; botonHabilitado=true	**Aprobada**
P20	Los datos originales no se modifican tras todas las pruebas	SHA-256 de app.js, index.html y data/citas_ficticias.json antes/después; FALLBACK_DOCTORS	archivosIguales=true; fallbackDoctorsIguales=true	archivosIguales=true; fallbackDoctorsIguales=true	**Aprobada**

Evidencia de ejecución

Primero se muestran las capturas generales (definición de esperados y ejecución). Después hay una captura por cada caso, de P01 a P20, con el ID del caso visible en la parte superior de la imagen. Los casos de interfaz son capturas reales de index.html en Chromium; los casos de lógica pura (P04 a P11) muestran la llamada a fallbackAvailability ejecutada en la consola del navegador, y P20 los hashes antes y después.

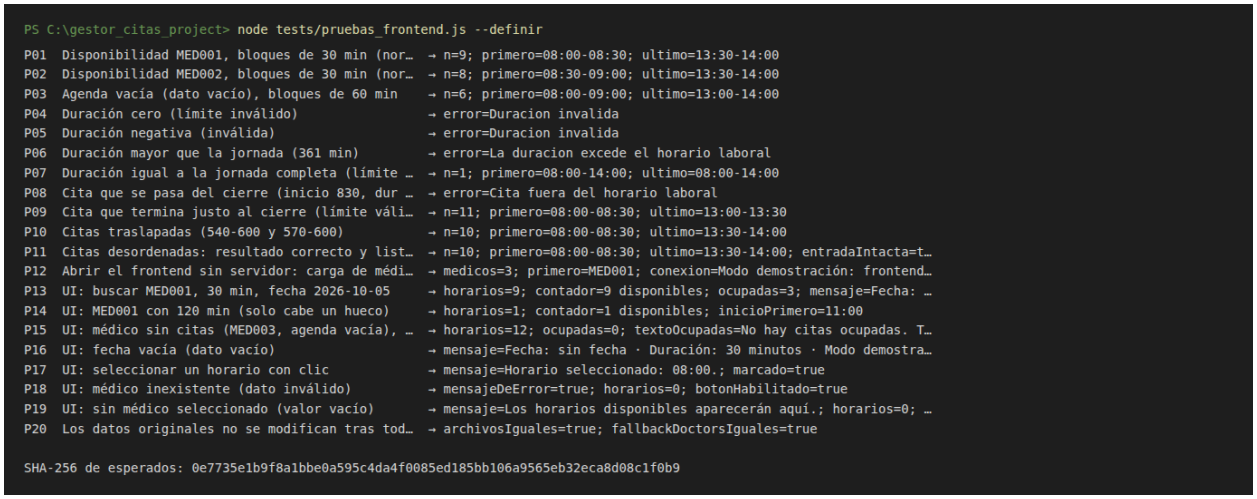


Figura 1. *Fase 1: resultados esperados definidos antes de ejecutar (comando --definir).*

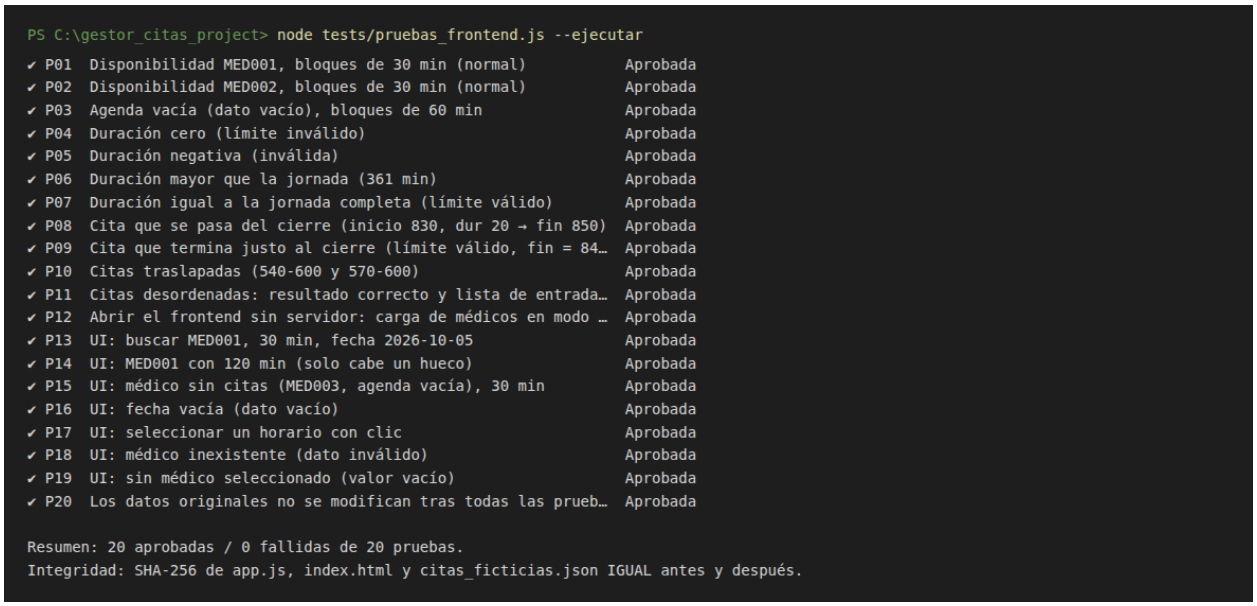


Figura 2. *Fase 2: ejecución de las 20 pruebas y comparación contra lo esperado (comando --ejecutar).*

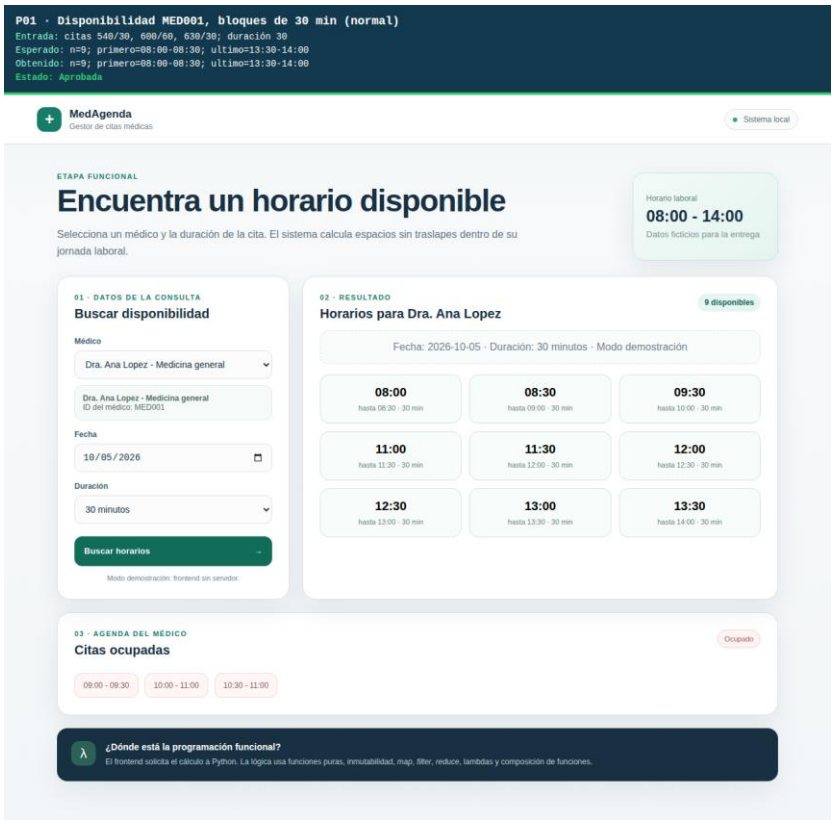


Figura 3. *Caso P01 – Disponibilidad MED001, bloques de 30 min (normal)*
Estado: Aprobada

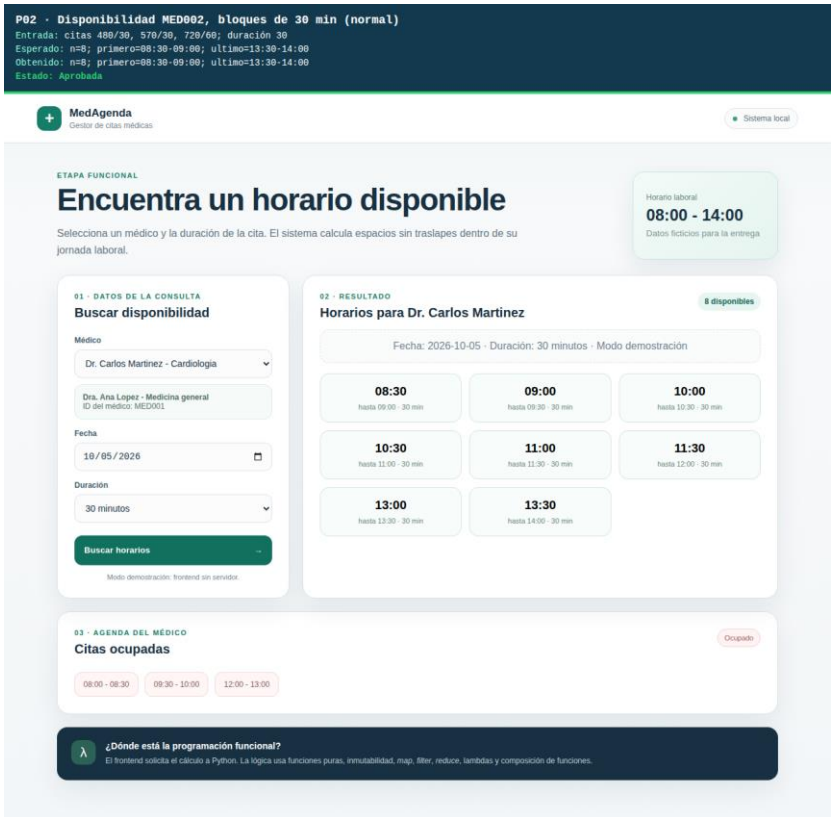


Figura 4. *Caso P02 – Disponibilidad MED002, bloques de 30 min (normal)*
Estado: Aprobada

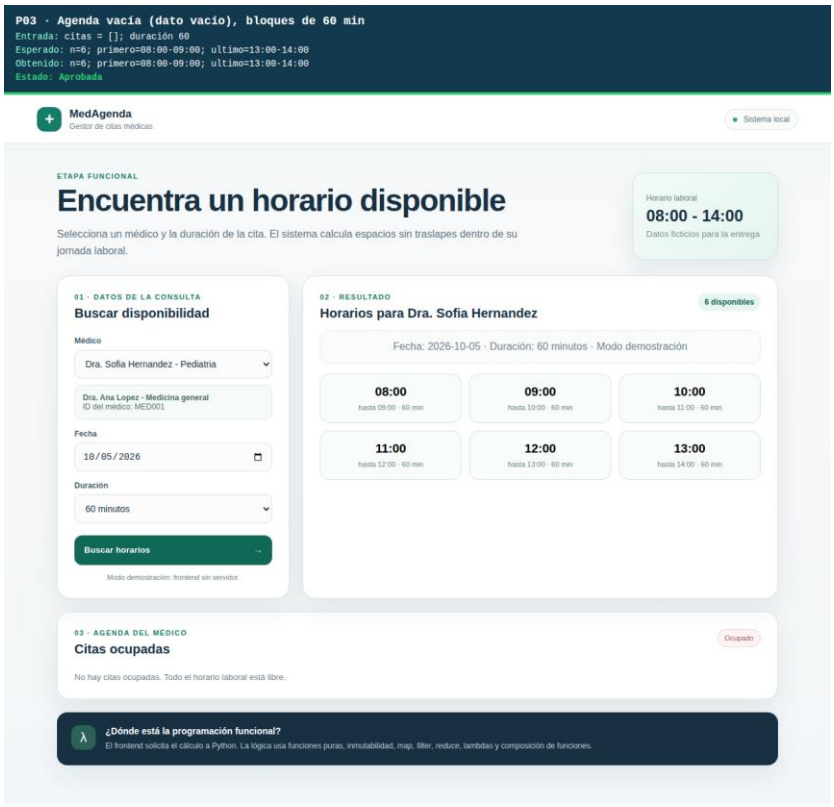


Figura 5. *Caso P03 – Agenda vacía (dato vacío), bloques de 60 min*
Estado: Aprobada

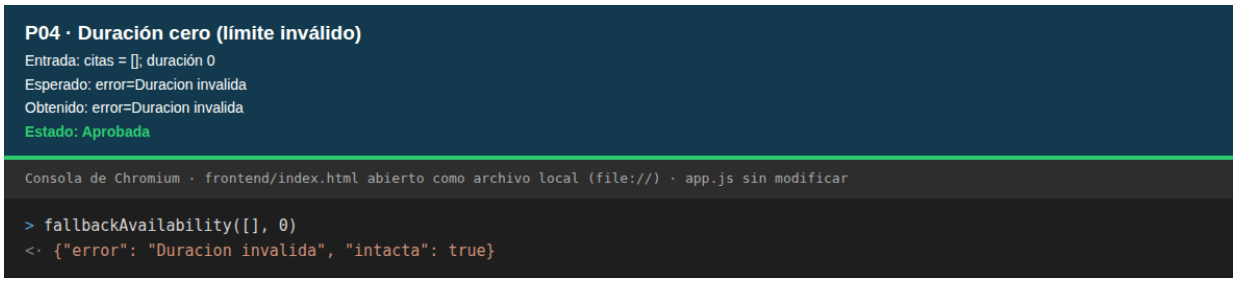


Figura 6. *Caso P04 – Duración cero (límite inválido)*
Estado: Aprobada

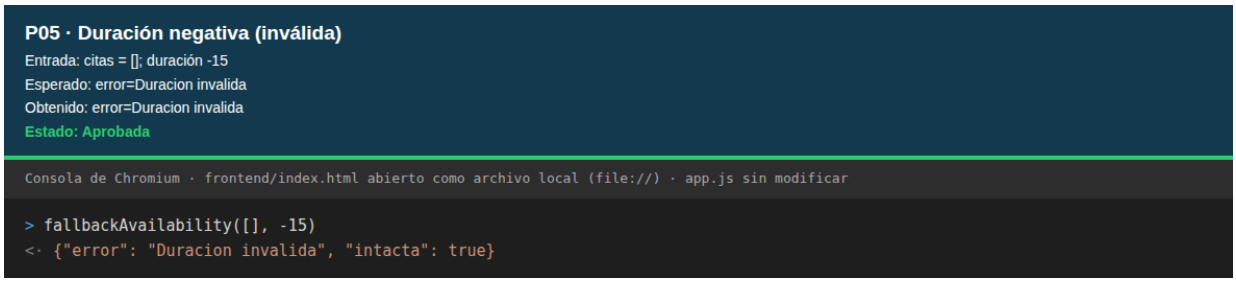


Figura 7. *Caso P05 – Duración negativa (inválida)*
Estado: Aprobada



Figura 8. *Caso P06 – Duración mayor que la jornada (361 min)*
Estado: Aprobada



Figura 9. *Caso P07 – Duración igual a la jornada completa (límite válido)*
Estado: Aprobada



Figura 10. *Caso P08 – Cita que se pasa del cierre (inicio 830, dur 20 → fin 850)*
Estado: Aprobada

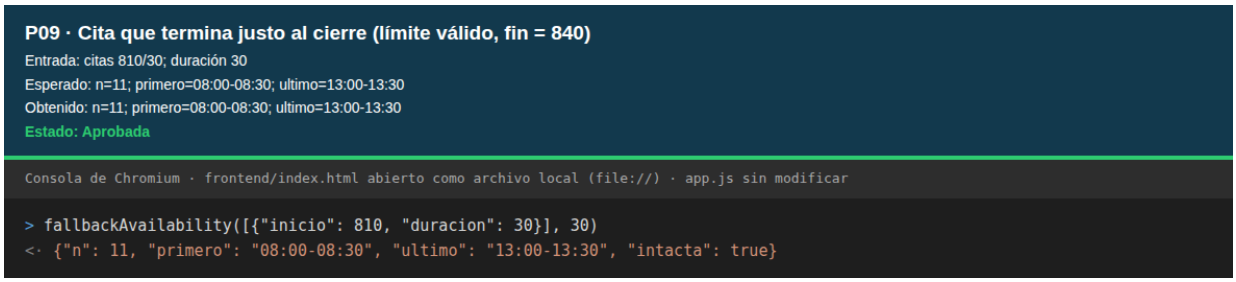


Figura 11. *Caso P09 – Cita que termina justo al cierre (límite válido, fin = 840)*
Estado: Aprobada

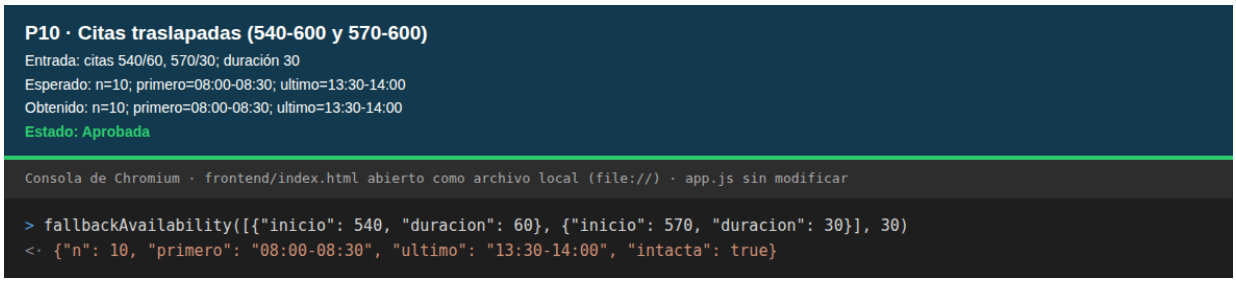


Figura 12. *Caso P10 – Citas traslapadas (540-600 y 570-600)*
Estado: Aprobada

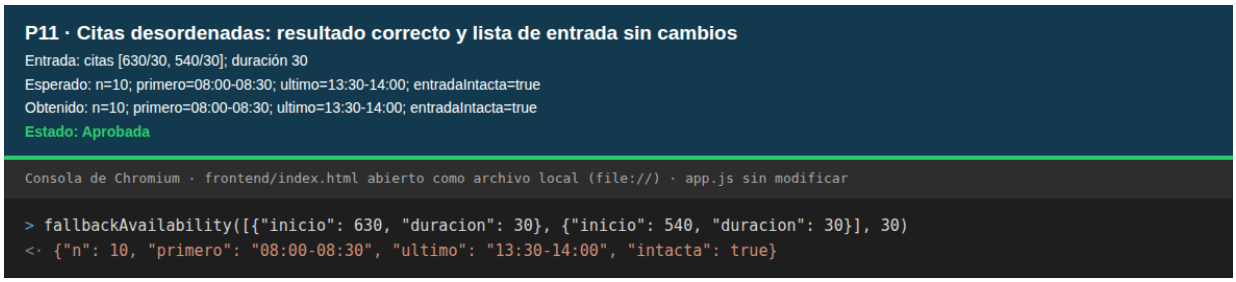


Figura 13. *Caso P11 – Citas desordenadas: resultado correcto y lista de entrada sin cambios*
Estado: Aprobada

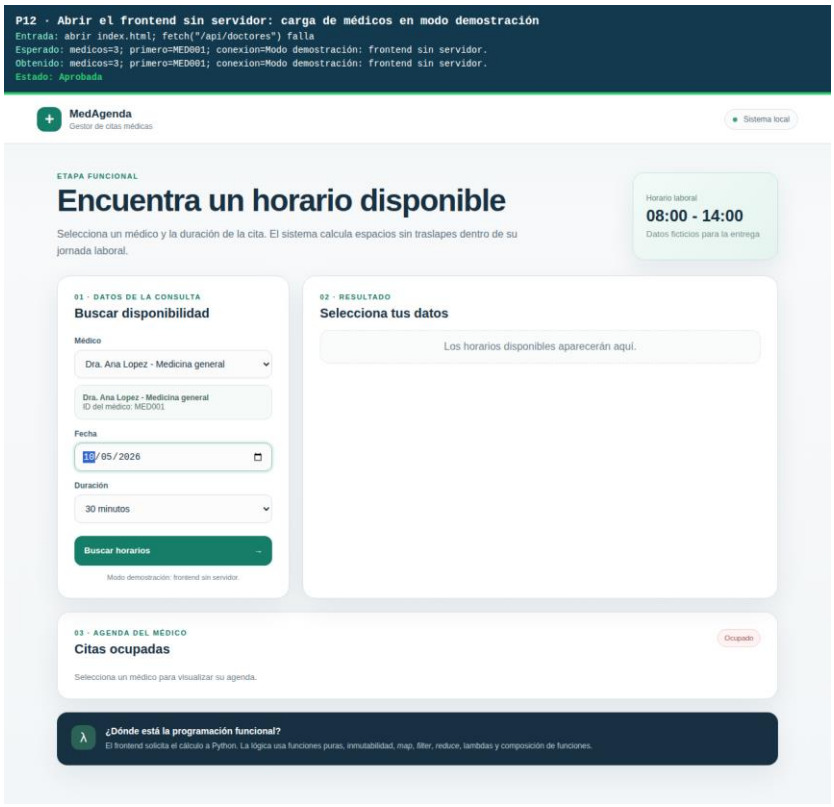


Figura 14. *Caso P12 – Abrir el frontend sin servidor: carga de médicos en modo demostración*
Estado: Aprobada

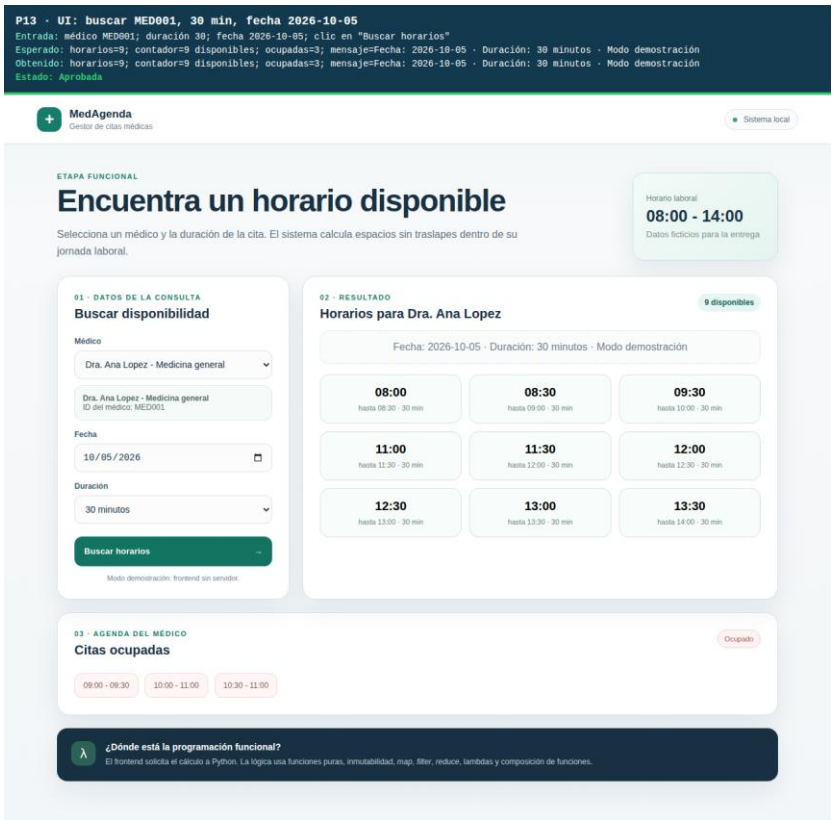


Figura 15. *Caso P13 – UI: buscar MED001, 30 min, fecha 2026-10-05*
Estado: Aprobada

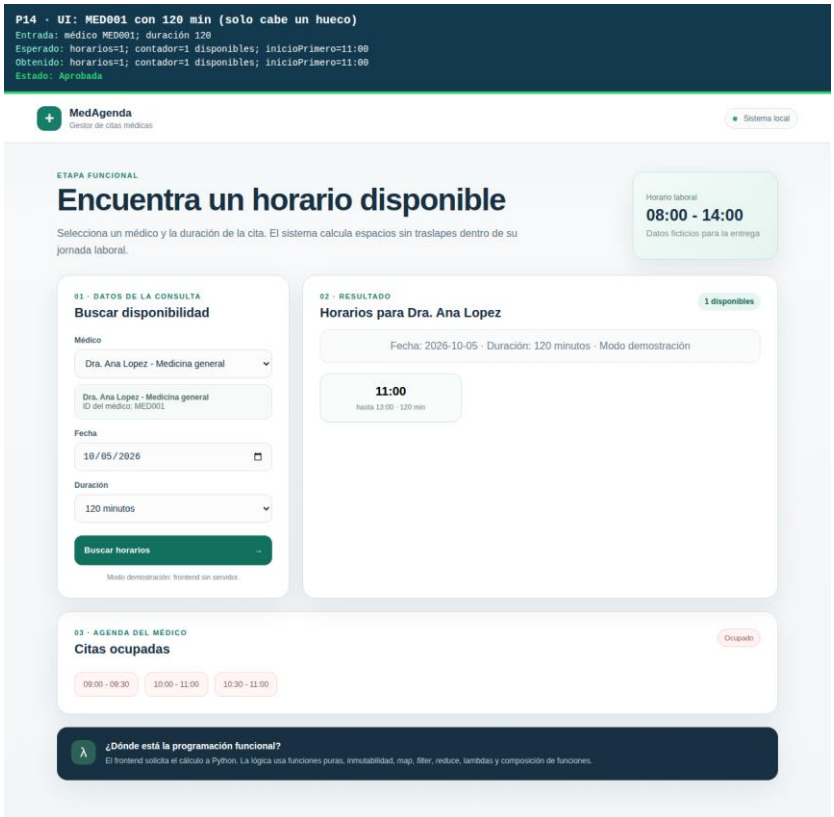


Figura 16. *Caso P14 – UI: MED001 con 120 min (solo cabe un hueco)*
Estado: Aprobada

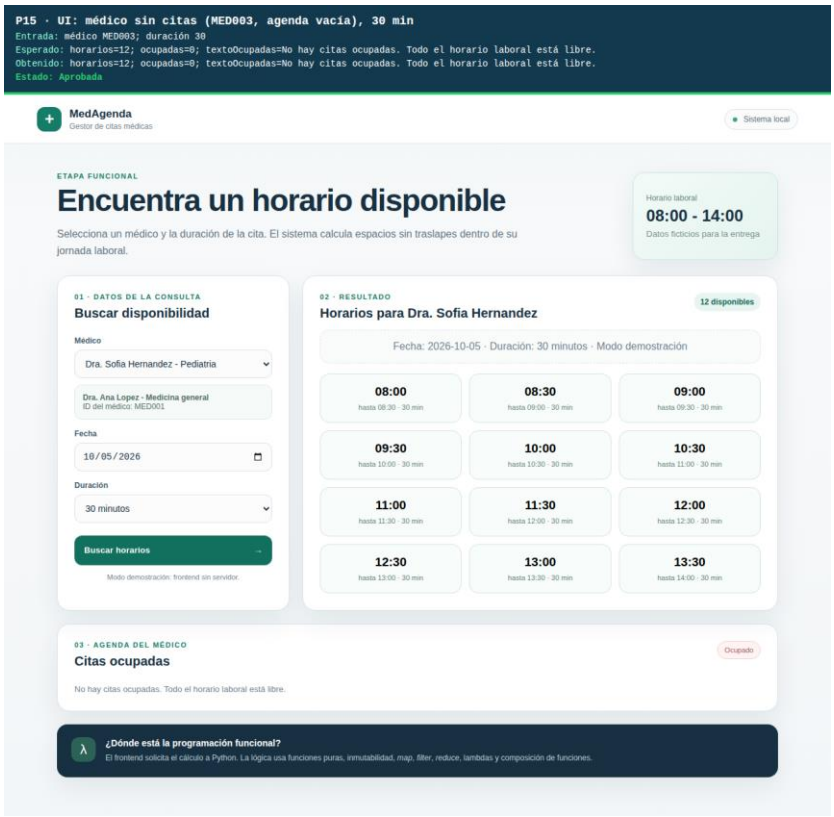


Figura 17. *Caso P15 – UI: médico sin citas (MED003, agenda vacía), 30 min*
Estado: Aprobada

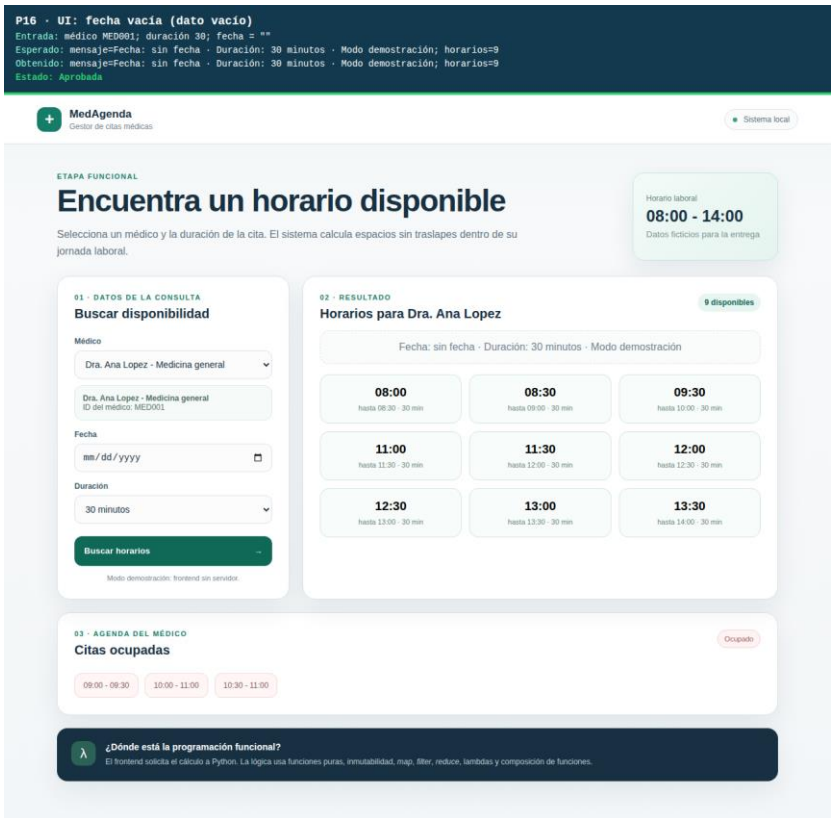


Figura 18. *Caso P16 – UI: fecha vacía (dato vacío)*
Estado: Aprobada

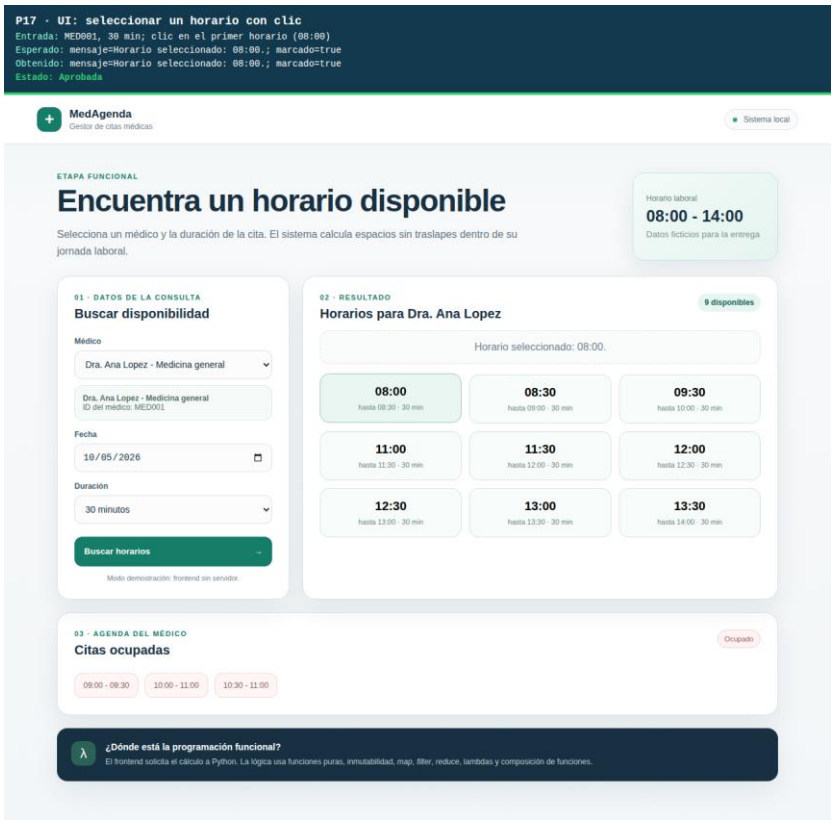


Figura 19. *Caso P17 – UI: seleccionar un horario con clic*
Estado: Aprobada

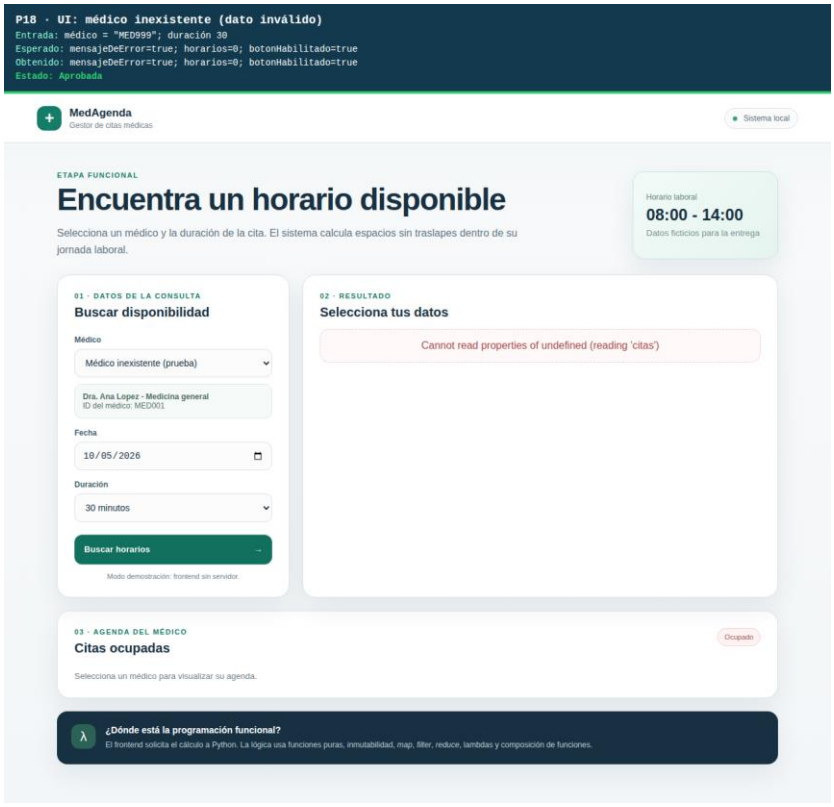


Figura 20. *Caso P18 – UI: médico inexistente (dato inválido)*
Estado: Aprobada

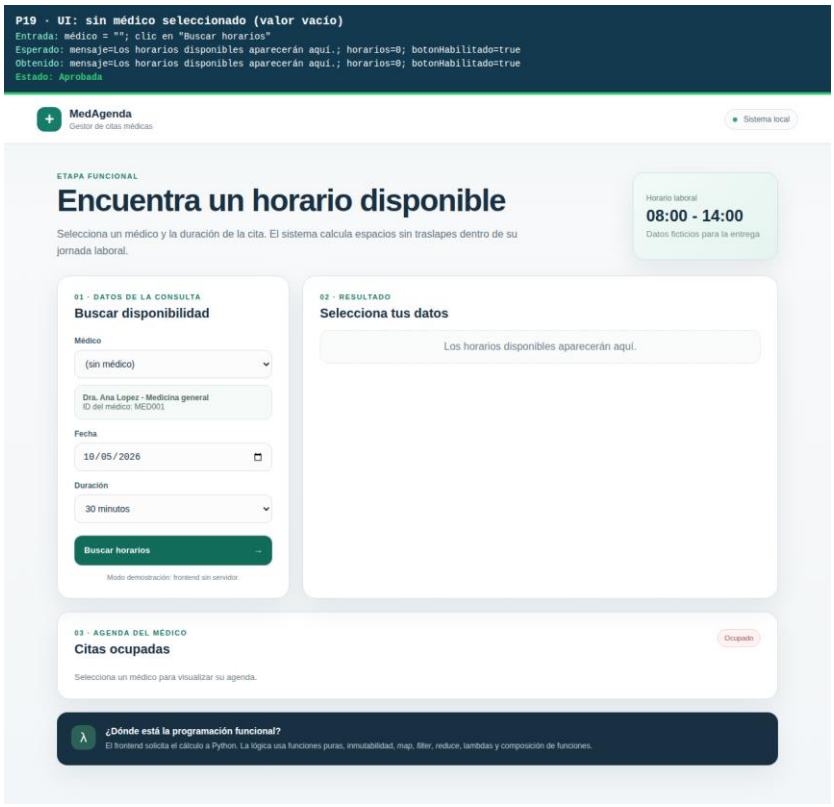


Figura 21. *Caso P19 – UI: sin médico seleccionado (valor vacío)*
Estado: Aprobada

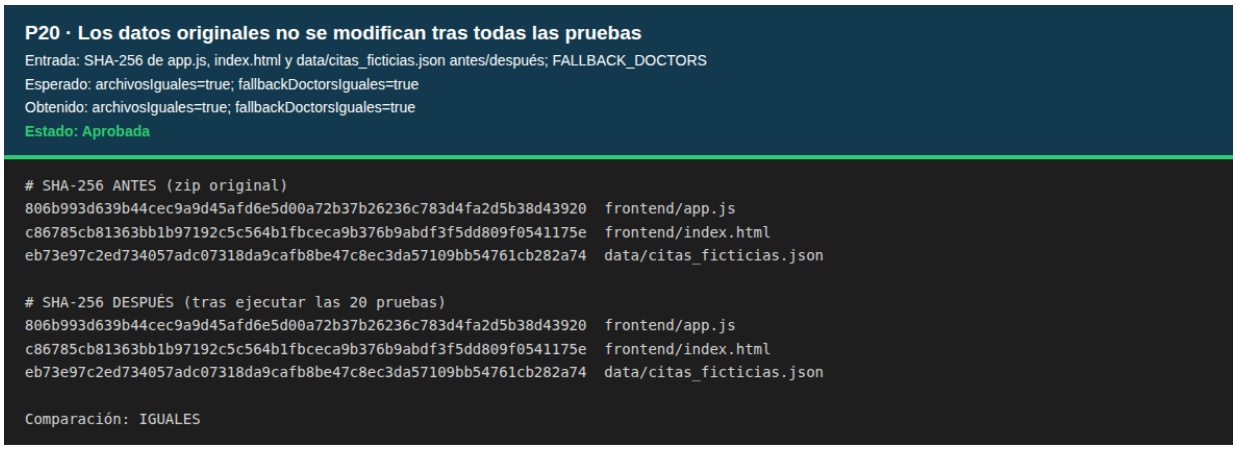


Figura 22. *Caso P20 – Los datos originales no se modifican tras todas las pruebas*
Estado: Aprobada

5.1 Resultado global

Las 20 pruebas resultaron aprobadas: El frontend calcula correctamente los horarios disponibles, rechaza duraciones y citas inválidas, maneja agendas vacías y datos incompletos, y no altera los datos originales.

6. Repositorio: https://github.com/mgKrisz10/Gestor_Hospital



7. Conclusiones

La funcionalidad de búsqueda de horarios disponibles quedó diseñada e implementada con un flujo que va desde los requerimientos y casos de uso hasta el pseudocódigo y el código en Python. La solución se apoya en inmutabilidad y composición funcional para transformar la agenda del médico en intervalos disponibles.

Agregar el frontend nos ayudó a ver el resultado de forma clara sin romper la separación del cálculo. La interfaz puede consumir el backend en Python o usar el modo de demostración con datos ficticios, mientras que la lógica principal sigue concentrada en el módulo funcional y se puede verificar con las 15 pruebas automatizadas.

Anexo A. Fragmento principal de implementación

```
def calcular_fin(cita: Cita) -> int:
    return cita.inicio + cita.duracion


def validar_cita(cita: Cita, horario: Intervalo) -> Resultado:
    if cita.duracion <= 0:
        return Error("Duracion invalida")
    if cita.inicio < horario.inicio or calcular_fin(cita) > horario.fin:
        return Error("Cita fuera del horario laboral")
    return Exito(cita)


def cita_a_intervalo(cita: Cita) -> Intervalo:
    return Intervalo(cita.inicio, calcular_fin(cita))


def ordenar_por_inicio(intervalos: Sequence[Intervalo]) -> Tuple[Intervalo, ...]:
    return tuple(sorted(intervalos, key=lambda intervalo: intervalo.inicio))


def fusionar(intervalos: Sequence[Intervalo]) -> Tuple[Intervalo, ...]:
    def acumular(
        acumulados: Tuple[Intervalo, ...], actual: Intervalo
    ) -> Tuple[Intervalo, ...]:
        if not acumulados:
```

```
            return (actual,)
        ultimo = acumulados[-1]
        if actual.inicio <= ultimo.fin:
            combinado = Intervalo(ultimo.inicio, max(ultimo.fin, actual.fin))
            return acumulados[:-1] + (combinado,)
        return acumulados + (actual,)
    return reduce(acumular, intervalos, ())


def calcular_huecos(
    ocupados: Sequence[Intervalo], horario: Intervalo
) -> Tuple[Intervalo, ...]:
    puntos = (
        Intervalo(horario.inicio, horario.inicio),
        *ocupados,
        Intervalo(horario.fin, horario.fin),
    )
    pares = zip(puntos[:-1], puntos[1:])
    candidatos = tuple(
        map(lambda par: Intervalo(par[0].fin, par[1].inicio), pares)
    )
    return tuple(filter(lambda hueco: hueco.fin > hueco.inicio, candidatos))


def dividir_hueco(hueco: Intervalo, duracion: int) -> Tuple[Intervalo, ...]:
    cantidad = (hueco.fin - hueco.inicio) // duracion
    return tuple(
        map(
            lambda k: Intervalo(
                hueco.inicio + k * duracion,
                hueco.inicio + (k + 1) * duracion,
            ),
            range(cantidad),
        )
    )


def formatear_hora(minutos: int) -> str:
    horas, mins = divmod(minutos, 60)
    return f"{horas:02d}:{mins:02d}"


def componer(*funciones: Callable) -> Callable:
    def pipeline(valor):
        return reduce(lambda actual, funcion: funcion(actual), funciones, valor)
    return pipeline


def buscar_disponibilidad(
    citas: Sequence[Cita], horario: Intervalo, duracion: int
```

```
) -> Resultado:
    """Calcula bloques disponibles sin modificar las citas de entrada."""
    if duracion <= 0:
        return Error("Duracion invalida")
    if duracion > horario.fin - horario.inicio:
        return Error("La duracion excede el horario laboral")

    validaciones = tuple(map(lambda cita: validar_cita(cita, horario), citas))
    errores = tuple(filter(lambda r: isinstance(r, Error), validaciones))
    if errores:
        return errores[0]

    pipeline = componer(
        lambda cs: tuple(map(cita_a_intervalo, cs)),
        ordenar_por_inicio,
        fusionar,
        lambda ocupadas: calcular_huecos(ocupadas, horario),
        lambda huecos: tuple(
            chain.from_iterable(
                map(lambda hueco: dividir_hueco(hueco, duracion), huecos)
            )
        ),
    )
    return Exito(pipeline(tuple(citas)))


def intervalo_a_dict(intervalo: Intervalo) -> dict:
    return asdict(intervalo)


def resultado_a_dict(resultado: Resultado) -> dict:
    if isinstance(resultado, Error):
        return {"ok": False, "error": resultado.mensaje}
    return {
        "ok": True,
        "intervalos": [intervalo_a_dict(i) for i in resultado.valor],
        "horario": intervalo_a_dict(HORARIO_LABORAL),
    }
```

Continuación de la Entrega Parcial 1 · Etapa Funcional

8. Segundo parcial: actualización y objetivo del MVP

Aclaración de alcance. Las secciones 1 a 7 y el Anexo A del reporte original corresponden al primer parcial y contienen requisitos generales previstos. Para las tres pruebas aleatorias del segundo parcial solo se presentan como implementadas las operaciones declaradas en este capítulo; el resto es trabajo futuro.

En el primer parcial se programó la búsqueda de horarios disponibles utilizando funciones puras, estructuras inmutables y datos ficticios. Para esta segunda entrega se aprovecha ese cálculo y se implementa un ciclo pequeño de reserva y cancelación de citas con información persistente.

8.1 Alcance implementado: tres operaciones

Operación	Qué se puede verificar en el demo
Consultar horarios libres	Elegir uno de los tres médicos, una fecha y duración de 15, 30, 45 o 60 minutos. Ver los bloques disponibles dentro de 08:00-14:00.
Agendar una cita	Seleccionar un bloque libre, escribir el nombre del paciente, confirmar y visualizar la nueva cita. No se acepta un empalme con otra cita del mismo médico y fecha.
Cancelar una cita	Cancelar una cita programada desde la tabla. Su estado cambia a «cancelada» y el horario vuelve a estar libre.

8.2 Alcance de la evaluación en laboratorio

Las tres pruebas del laboratorio serán seleccionadas por la profesora de manera aleatoria a partir de las funciones declaradas en el reporte. Por eso, las tres operaciones anteriores constituyen el alcance evaluable; la tabla de citas, el catálogo de médicos y el almacenamiento SQLite solo permiten completar y comprobar esas operaciones.

No se considera parte de este MVP: reprogramación, alta o edición de médicos, edición independiente de pacientes, cuentas de usuario, permisos por rol, notificaciones e historial clínico.

8.3 Reglas del proceso

- La jornada laboral de la demostración es de 08:00 a 14:00; toda cita debe quedar completamente dentro de ese intervalo.
- La fecha debe existir en el calendario y escribirse en formato AAAA-MM-DD.
- No se permiten dos citas activas traslapadas para el mismo médico y la misma fecha.
- Cancelar marca el registro como cancelado y libera su bloque; no lo elimina del historial de la tabla.
- Los datos se guardan en una base SQLite local. Al volver a abrir el sistema, las citas siguen registradas.

9. Diseño del Producto Mínimo Viable

9.1 Usuario y flujo principal

El actor principal es el personal de recepción. Primero selecciona médico, fecha y duración; después elige un bloque disponible y registra el nombre del paciente. Finalmente puede consultar la tabla para confirmar o cancelar la reserva.

Paso	Entrada o acción	Resultado
1	Seleccionar médico, fecha y duración	Se consulta la disponibilidad del día.
2	Elegir un horario libre	Queda seleccionado el intervalo de la reserva.
3	Indicar nombre y confirmar	El servidor valida y registra una cita programada.
4	Volver a buscar el horario	El bloque reservado aparece ocupado.
5	Cancelar la cita desde la tabla	El bloque se libera y el estado cambia a cancelada.

9.2 Requisitos cubiertos

Se implementan R04 (mostrar disponibilidad), R05 (agendar), R06 (impedir conflictos) y R07 (cancelar). El nombre del paciente se almacena durante el agendado como una parte mínima de R01. Los demás requisitos del primer informe no se presentan como implementados.

9.3 Estructura de los datos

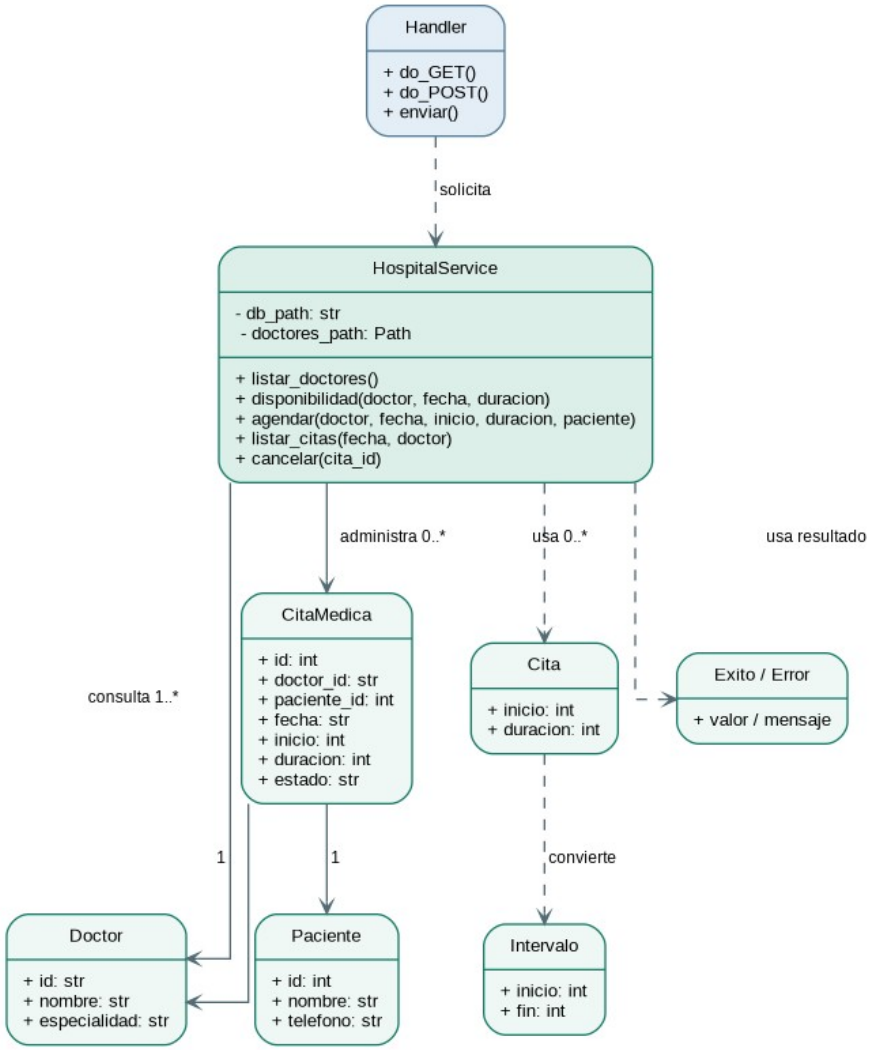
Entidad	Información	Uso
Doctor	ID, nombre, especialidad	Seleccionar el profesional de la consulta.
Paciente	ID, nombre, teléfono opcional	Vincular la reserva con un nombre ficticio.
Cita médica	ID, doctor, paciente, fecha, inicio, duración, estado	Distinguir reservas activas y canceladas.

9.4 Validaciones importantes

Cuando se confirma una cita, el servidor vuelve a consultar las reservas activas antes de guardarla. Así se evita que dos personas ocupen accidentalmente el mismo horario si ambas habían visto libre ese bloque. El solapamiento se verifica comparando inicio y fin de intervalos, no solamente la hora inicial.

10. Diagrama de clases del MVP

Las clases de dominio Doctor, Paciente y CitaMedica representan los datos. HospitalService concentra las operaciones del prototipo y utiliza el cálculo funcional heredado de la primera entrega. Handler comunica la interfaz web con el servicio.



El diagrama está disponible también en PNG, SVG y DOT dentro de la carpeta docs del proyecto. Las funciones de reprogramación no forman parte de este diseño.

11. Implementación del MVP

La implementación usa Python 3 con bibliotecas estándar, JavaScript y HTML/CSS. Se inicia con app.py, que levanta un servidor local. La persistencia se realiza con SQLite, sin necesidad de instalar servicios externos.

Archivo	Función
app.py	Arranque del servidor y apertura del navegador.
src/gestor_citas.py	Cálculo funcional de intervalos y huecos del primer parcial.
src/mvp.py	HospitalService, modelos, validaciones y consultas SQLite.
src/servidor.py	API HTTP y publicación de archivos de la interfaz.
frontend/mvp.html, mvp.js, mvp.css	Formulario para consultar, agendar y cancelar.
data/citas_ficticias.json	Catálogo inicial de los tres médicos.
tests/test_mvp.py	Pruebas del ciclo de cita y reglas relacionadas.

11.1 API utilizada

Método	Ruta	Resultado
GET	/api/mvp/doctores	Catálogo de médicos.
GET	/api/mvp/disponibilidad	Bloques libres para médico, fecha y duración.
GET	/api/mvp/citas	Tabla de citas registradas.
POST	/api/mvp/citas	Alta de cita y paciente.
POST	/api/mvp/citas/{id}/cancelar	Cancelación de cita programada.

11.2 Persistencia y datos de demostración

La base data/hospital.sqlite3 se genera al iniciar por primera vez. Las tablas son doctores, pacientes y citas. Para una demostración limpia, puede cerrarse el servidor y eliminarse solo ese archivo. El JSON se utiliza para el catálogo de médicos; las citas ficticias del primer informe no se importan como reservas activas en la nueva base.

12. Pruebas y criterios de aceptación

En el entorno local se ejecutaron 33 pruebas automatizadas de unittest: 15 del cálculo funcional previo y 18 de las operaciones del MVP y sus reglas. Las 33 finalizaron correctamente. La profesora puede elegir tres entradas y situaciones diferentes durante el laboratorio; la siguiente tabla sirve para practicar variantes, no pretende anticipar cuáles escogerá.

Operación	Variante posible	Respuesta esperada
Consultar	Médico sin reservas, duración 30 min	12 bloques en la jornada completa 08:00-14:00.
Consultar	Médico sin reservas, duración 60 min	6 bloques en la jornada completa.
Agendar	Cita libre a las 08:00 por 30 min	Alta correcta y bloque ocupado.
Agendar	Misma fecha, médico y horario reservado	Rechazo; no se agrega una segunda cita.
Agendar	Empalme parcial, p. ej. 10:00-11:00 y 10:30-11:00	Rechazo de la segunda cita.
Agendar	Misma hora con diferente médico o fecha	Operaciones independientes y válidas.
Agendar	Fecha imposible, nombre vacío o cita fuera de jornada	Mensaje de validación, sin nueva reserva.
Cancelar	Cancelar una cita activa	Pasa a cancelada, el bloque vuelve a quedar libre.
Cancelar	Cancelar nuevamente la misma cita	Rechazo sin duplicar o alterar registros.

12.1 Cómo se verificó

Las pruebas utilizan bases SQLite temporales para no alterar los datos de una demostración. Se evalúa también la conservación de registros al crear una nueva instancia del servicio y el bloqueo de reservas concurrentes al mismo horario. Los resultados no sustituyen la ejecución presencial de las tres pruebas.

12.2 Precaución al demostrar empalmes

La pantalla deja de ofrecer los bloques ocupados; por ello el rechazo del servidor puede demostrarse mediante la prueba automatizada test_05_rechazo_cita_duplicada o test_06_rechazo_empalme_parcial, sin modificar la interfaz.

13. Ejecución del proyecto

1. Descargar los archivos del proyecto y abrir la carpeta que contiene app.py en Visual Studio Code.

2. Abrir una terminal y ejecutar: python app.py. En Windows también puede utilizarse INICIAR.bat.

3. Abrir en el navegador la dirección local mostrada en terminal, por ejemplo http://127.0.0.1:8000/.

4. Elegir un médico, fecha y duración, buscar horarios, seleccionar uno libre y registrar un paciente ficticio.

5. Comprobar que la cita aparece en la tabla. Si se cancela, el bloque vuelve a estar disponible.

6. Para ejecutar las pruebas: python -m unittest discover -s tests -v.

14. Conclusiones del segundo parcial

En esta etapa se transformó la consulta funcional de horarios del primer parcial en un MVP pequeño que registra citas y permite cancelarlas. La selección de tres operaciones reduce la complejidad de la revisión y permite comprobar cada comportamiento con distintos datos. El código, la interfaz y el diagrama se ajustaron para que el reporte describa únicamente lo que se presenta en la demostración.

15. Repositorio y materiales de entrega

Repositorio del proyecto: https://github.com/mgKrisz10/Gestor_Hospital

El repositorio es la referencia oficial de código de la asignatura. Para presentar el segundo parcial, deben estar publicados allí app.py, src/mvp.py, src/servidor.py, frontend/mvp.html, frontend/mvp.js, frontend/mvp.css y tests/test_mvp.py, además de este informe en PDF. La copia local incluida en el ZIP tiene esos componentes para su integración.

16. Limitaciones

Este es un prototipo académico local sin autenticación ni medidas para operar con información clínica real. No se han implementado funciones de administración médica o notificaciones. Se recomienda utilizar siempre nombres y teléfonos ficticios al demostrar.

Continuidad documental: se conservaron íntegramente las 25 páginas del reporte del primer parcial y se agregó una nota de alcance al comienzo del PDF actualizado. Aquellos requerimientos proyectados que no se describen como implementados en este bloque no forman parte del MVP.